

PAPER DEEP DIVE · ARXIV 2509.24527

Dreamer 4

Training Agents Inside of Scalable World Models

Danijar Hafner*, Wilson Yan*, Timothy Lillicrap — Google DeepMind
29 September 2025 · danijar.com/project/dreamer4

Presenter _____ Date _____ ~45–60 min incl. discussion

Right: frames generated by the Dreamer 4 world model — an imagined rollout, a human playing inside the model, and the robot-arm model.



Agenda

- 01 Background** why world models, what made scaling them hard, the offline diamond challenge
- 02 Method** tokenizer, dynamics transformer, shortcut forcing, agent tokens, imagination training (PMPO)
- 03 Experiments** real-time human play-testing, diamond challenge, ablations, learning from unlabeled video, robotics
- 04 Discussion** limitations, critical reading, future work, takeaways
- 05 Appendix** hyper-parameters, full tables, glossary

HOW TO READ THIS DECK

Equations are kept to the minimum needed for intuition; the appendix has the exact settings.

Every number is from the paper's tables (Table 1, 2, 7, 8) or the authors' talks — sources are on each slide.

Figures © the authors (arXiv 2509.24527 / danijar.com), reproduced for discussion.

Suggested pacing: Parts 1–2 ≈ 25 min, Part 3 ≈ 15 min, Part 4 ≈ 10 min + Q&A.

Dreamer 4 in one slide

What the paper claims, and the evidence behind each claim

Claim	Evidence in the paper
First agent to obtain diamonds in Minecraft from offline data only	0.7 % of $1,000 \times 60$ -min episodes reach a diamond; 29 % reach an iron pickaxe (vs 11 % for a Gemma-3 VLA, 0.6 % for BC) — Table 7
A world model that simulates Minecraft object interactions and menus accurately, in real time on 1 GPU	Human play-testers solve 14/16 interaction tasks inside it at 21 FPS with a 9.6 s context (Oasis: 5/16, Lucid-v1: 0/16) — Table 1
Shortcut forcing + an efficient transformer make this fast	4 sampling steps instead of 16–64; FVD 306 $\rightarrow$ 57 across the design cascade while going from 0.8 to 21 FPS — Table 2, Fig. 8
Action conditioning can be learned from few labels	100 h of labeled actions $\approx$ 85 % PSNR / 100 % SSIM of using all 2,541 h; actions transfer to Nether/End seen only unlabeled — Fig. 7
$\sim 100\times$ less data than VPT, no environment interaction	2,541 h contractor video vs VPT's 270 K h web video + online RL — Table 3

PART 1

Background & motivation

World models, learning in imagination, and why Minecraft from offline data is a hard test

01

Why world models? Learning in imagination

The Dreamer lineage: same idea, increasingly capable simulators

- **World model:** a learned simulator that predicts what happens next given the current situation and an action
- **Imagination training:** the policy is optimised on rollouts generated *by the model*, not by the real environment — cheap, parallel, safe
- **Why it matters now:** robots and other real-world systems cannot afford millions of trial-and-error interactions; video on the internet is abundant
- **Dreamer 4's bet:** if the simulator is accurate enough about object interactions, RL inside it can go beyond what pure imitation learns



Imagined rollout decoded for visualisation, with the agent's reward and value estimates (project page, 'Gather wood').

Dreamer (2019)

latent imagination on DM Control

DreamerV2 (2020)

discrete latents, Atari

DreamerV3 (2023/Nature 2025)

one config for 150+ tasks; diamonds with online RL

Dreamer 4 (2025)

transformer diffusion WM; diamonds offline; real-time on 1 GPU

What made scalable world models hard before

Three requirements pulled in different directions

01

Fidelity of interactions

Video generators look good but get game mechanics wrong: inventories, crafting menus, breaking blocks, placing a boat and riding it.

Prior Minecraft world models (Oasis, Lucid-v1, MineWorld) fail most such tasks.

02

Speed

Imagination RL needs millions of generated frames — and a human tester needs ≥ 20 FPS.

Diffusion transformers spend tens of denoising steps per frame: 0.8 FPS for the baseline in this paper.

03

Memory & drift

Autoregressive video drifts: small errors feed back into the context.

Short contexts (1–2 s in prior models) forget what is behind the player.

- **Dreamer 3's RSSM** is fast ($\sim 1,000\times$ faster than a diffusion transformer) but operates at 64×64 pixels and needed abstract inventory state and crafting actions — not raw mouse & keyboard
- **Dreamer 4's answer:** a diffusion transformer that needs only 4 sampling steps (shortcut forcing), an architecture tuned for long context at low cost, and training tricks that keep long rollouts stable (x-prediction, noised context)

The test bed: the offline diamond challenge

No environment interaction at all — learn everything from 2,541 hours of human gameplay video

- **Data:** OpenAI VPT contractor recordings (subsets 6–10), 2,541 h at 360×640, 20 FPS, with mouse & keyboard labels; 90/10 train/eval split by 5-minute chunk
- **Actions:** 23 binary keys + 121-way mouse class (11×11 foveated bins, VPT convention) — no abstract 'craft' actions
- **Evaluation:** 1,000 fresh random worlds, empty inventory, 60 min each (72,000 actions at 20 FPS); a fixed prompt sequence walks the agent through the 12-item tech tree
- **Why it is hard:** diamonds sit ~24,000 actions deep for a human; rewards are sparse; offline data covers the late tech tree thinly; every skill (chop → craft → mine → smelt) must chain without a single trial in the real game
- **Baselines, all on the same data:** VPT fine-tuned, BC, BC without task input, a VLA built on Gemma 3, and 'WM + BC' (Dreamer 4 without imagination RL)



Diamond ore, as simulated by the world model during a human 'mine diamonds' session (project page).

THE 12 MILESTONES (TABLE 5)

log → planks → crafting table → stick → wooden pickaxe → cobblestone
→ stone pickaxe → iron ore → furnace → iron ingot → iron pickaxe →
diamond

PART 2

Method

One transformer, three phases: world-model pretraining → agent fine-tuning → imagination training

02

Overview: three phases, one transformer

Algorithm 1 in the paper

Phase 1

World-model pretraining

- Train the causal tokenizer (masked autoencoding), then freeze it
- Train the interactive dynamics model on tokenized video $\pm$ actions with shortcut forcing
- Inputs: all 2,541 h, labeled or not

→

Phase 2

Agent fine-tuning

- Insert agent tokens with task embedding; add policy & reward heads
- Behavioural cloning with multi-token prediction ($L = 8$) on task-relevant data
- Dynamics loss continues on uniform data so the model stays honest

→

Phase 3

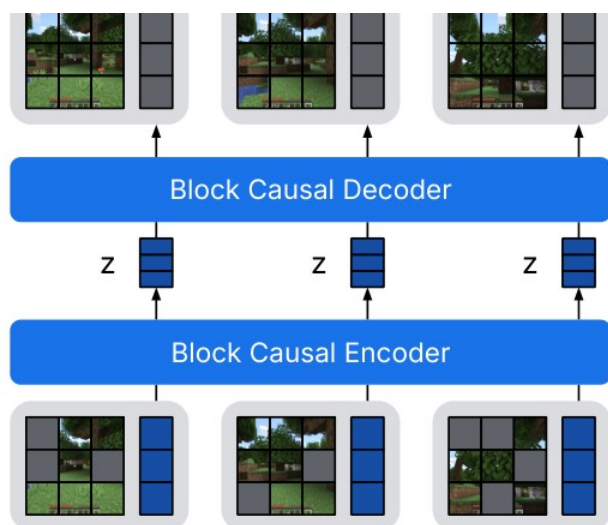
Imagination training

- Freeze the transformer; train policy + value heads only
- Roll out the policy inside the world model, reward from the learned reward head
- PMPO with reverse-KL to the frozen BC policy; $\gamma = 0.997$

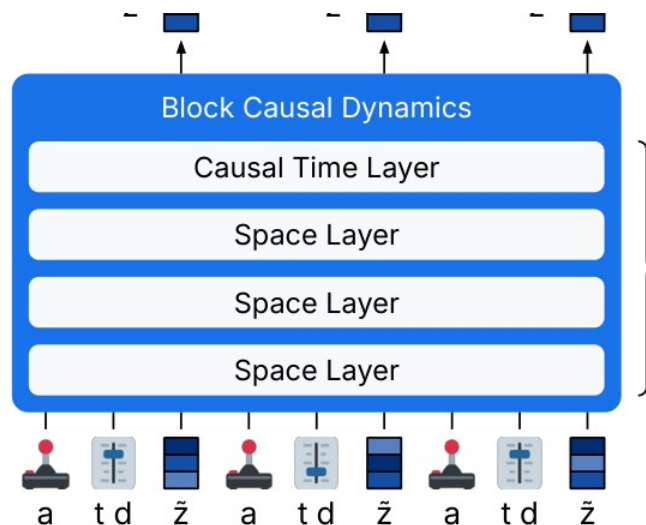
- **Key design choice:** the same block-causal transformer serves as tokenizer, dynamics model and agent backbone; policy and value are tiny heads on 'agent tokens'
- **Data mix in phases 2-3:** 50 % uniform sequences / 50 % task-relevant; BC loss only on relevant data, dynamics loss only on uniform data ('to avoid optimistic generations')

World-model design

Figure 2: causal tokenizer + interactive dynamics model, one shared block-causal transformer



(a) Causal Tokenizer



(b) Interactive Dynamics

- **Total 2 B parameters:** ≈ 400 M tokenizer + 1.6 B dynamics
- **Frames:** 360×640 , zero-padded to $384 \times 640 \rightarrow 960$ patches of 16×16
- **Latents:** $N_z = 256$ tokens $\times$ 32 dims per frame (512×16 bottleneck, tanh)
- **Context:** 192 frames = 9.6 s at 20 FPS
- **Block-causal:** full attention within a frame, causal across frames $\rightarrow$ usable online, frame by frame
- **Transformer recipe:** pre-RMSNorm, RoPE, SwiGLU, QK-norm, logit soft-capping, register tokens

Left: causal tokenizer (masked patches $\rightarrow$ latent tokens $\rightarrow$ reconstruction). Right: interactive dynamics — interleaved action tokens a , latent tokens z , and per-frame signal level τ / step size d ; space layers with a causal time layer every 4th block. (Figure 2, arXiv 2509.24527)

The causal tokenizer

Compress each frame into 256 latent tokens without looking into the future

- **Masked autoencoder, not a VAE/VQ:** patches are dropped with ratio $\sim U(0, 0.9)$ during training; encoder sees the surviving patches plus learned latent tokens, decoder reconstructs all patches
- **Loss:** MSE + 0.2 · LPIPS (perceptual term keeps textures crisp)
- **Bottleneck:** 512×16 per frame with tanh squashing, reshaped to 256 tokens × 32 dims for the dynamics model — a *continuous* latent, no codebook
- **Causal in time:** frame t is encoded/decoded using only frames $\leq t \rightarrow$ the same tokenizer works during live interaction
- **Why MAE-style?** Inspired by MAETok (masked autoencoders make good tokenizers for diffusion models): masking regularises the latent space so the dynamics model has an easier, smoother target
- **Trained once, then frozen** — Hafner expects fully end-to-end training to be possible eventually

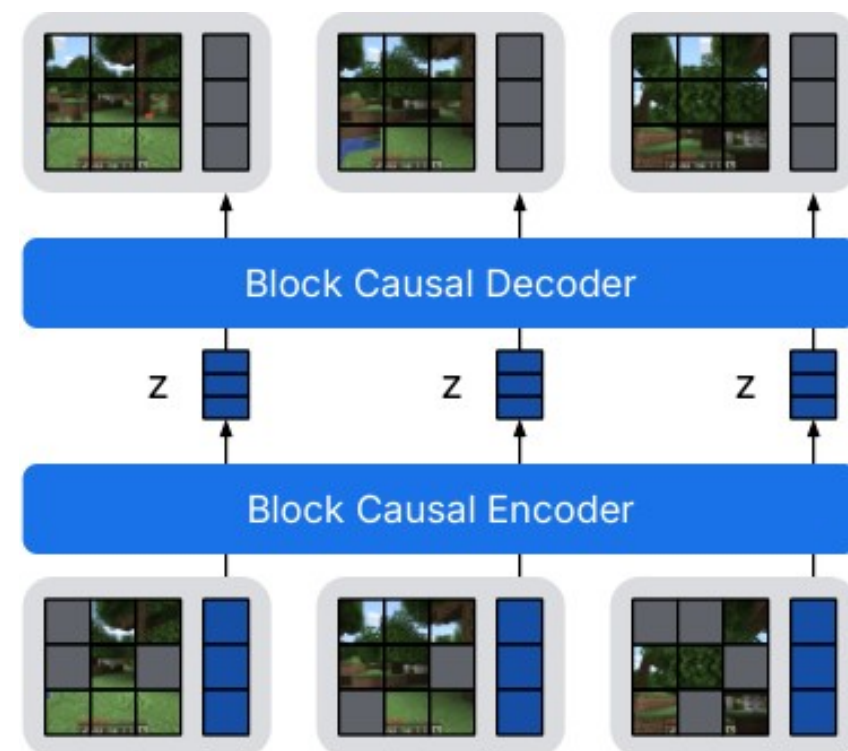


Figure 2a (arXiv HTML render).

The interactive dynamics model

Predict the next frame's latents from past latents, actions and the current noisy guess

- **Sequence layout:** per frame, action tokens (23 keys + mouse class, or a learned 'no action' embedding for unlabeled video), then 256 latent tokens, plus a token carrying signal level τ and step size d
- **Attention pattern:** space-only attention inside a frame in 3 of every 4 layers; a *causal time* layer every 4th layer sees the 192-frame context $\rightarrow$ most compute is spent on cheap per-frame attention
- **Efficiency tricks:** grouped-query attention, register tokens, alternating batch lengths (64-frame short batches interleaved with 256-frame long batches — long context is learned without paying for it every step)
- **Objective:** shortcut forcing — a diffusion/flow objective that (i) noises every frame independently and (ii) is conditioned on the step size so that 4 sampling steps suffice (next slides)
- **Inference:** 21 FPS at 640×360 on one H100 with $K = 4$ steps; context frames are kept slightly noisy ($\tau_{\text{ctx}} \approx 0.1$) so the model tolerates its own imperfections

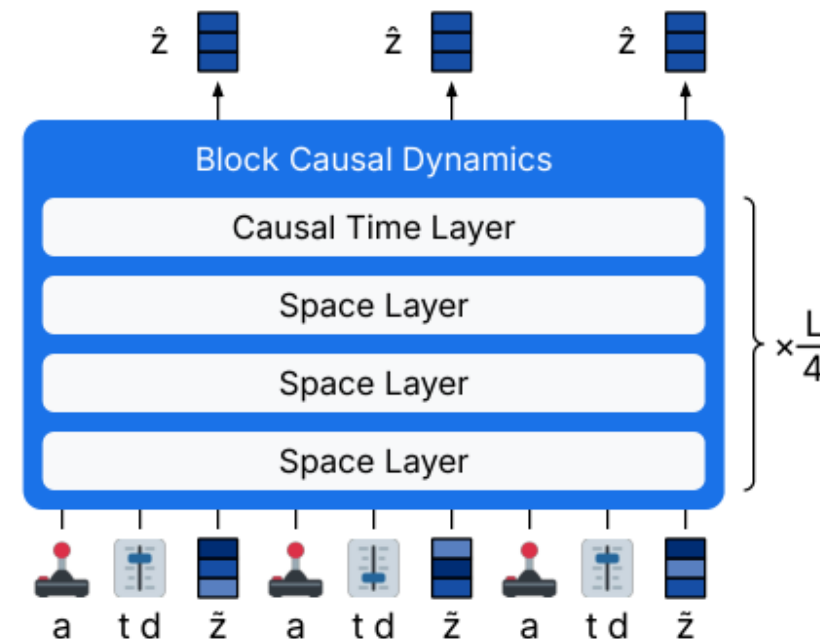


Figure 2b (arXiv HTML render).

Two-minute primer: flow matching, signal level τ , x vs v

Just enough background for shortcut forcing

Flow matching view of diffusion

Interpolate between noise ϵ and clean latent x : $z_\tau = \tau \cdot x + (1 - \tau) \cdot \epsilon$, $\tau \in [0, 1]$ (τ = signal level).

A network learns to move z_τ toward x ; sampling integrates from $\tau = 0$ (pure noise) to $\tau = 1$ (clean) in K steps.

Standard models need many small steps because the learned direction is only locally correct.

Diffusion forcing (Chen et al., 2024)

Give every frame in the sequence its *own* noise level instead of one level for the whole clip.

The model learns to predict a noisy future from a cleaner past $\rightarrow$ supports causal, frame-by-frame rollouts and keeping context frames slightly noisy at test time.

Cost: still tens of denoising steps per frame.

Two parameterisations of the same target

v -prediction: output the velocity $v = x - \epsilon$ (common in image/video diffusion).

x -prediction: output the clean latent x directly; the velocity follows as $(x - z_\tau)/(1 - \tau)$.

Mathematically equivalent — numerically very different when errors feed back autoregressively (slide 15).

Shortcut models (Frans et al., 2024)

Condition the network on the step size d as well as τ .

Self-consistency ('bootstrap') loss: one step of size $2d$ must equal two consecutive steps of size d .

Result: the model learns to take large, accurate steps $\rightarrow$ few-step or even one-step sampling.

Shortcut forcing: the training objective

Diffusion forcing + shortcut models, with three engineering choices that turn out to matter a lot

- **Per-frame noise:** each frame gets an independent signal level τ_t ; the model predicts frame t 's clean latent from the noisy frame t , past frames, and actions
- **Step-size conditioning:** the network also receives d ; for $d > d_{\min}$ the target is the *bootstrap* target — the result of two half-steps computed by the model itself (no gradient through the target)
- **Choice 1 — x-prediction:** the network outputs the clean latent, not the velocity
- **Choice 2 — loss in x-space:** the bootstrap target is formed in v -space (where two half-steps compose cleanly) but the loss is scaled back to x -space by $(1 - \tau)^2$
- **Choice 3 — ramp weight:** loss weight $w(\tau) = 0.9 \tau + 0.1$ emphasises high-signal (low-noise) levels, where fine detail is decided
- **Inference:** $K = 4$ steps per frame; context frames at $\tau_{\text{ctx}} \approx 0.1$; sampling remains stochastic $\rightarrow$ diverse imagined futures for RL

EFFECT OF EACH PIECE (TABLE 2, FVD $\downarrow$)

Diffusion forcing, $K = 64$ steps: 306 (0.8 FPS)

Same model, $K = 4$ steps: 875

+ shortcut objective: 329

+ x-prediction: 326

+ x-space loss: 151

+ ramp weight: 102

Full architecture but v -space prediction/loss: 124 vs 57

WHY THIS IS THE KEY CONTRIBUTION

4 steps $\approx$ the quality of 16–32 diffusion-forcing steps (Fig. 8) — roughly 16 $\times$ fewer network evaluations per frame, which is what makes real-time interaction and large-scale imagination RL feasible.

Why x-prediction matters for autoregressive world models

Hafner's explanation (TalkRL, Nov 2025) and the numbers

- **v-prediction carries the noise:** to output $v = x - \epsilon$ the network must reproduce the *exact* noise pattern of its input through every layer; small mistakes in that noise become errors in the sample
- **In autoregressive use these errors compound:** each generated frame becomes context for the next; velocity errors accumulate as drift, blur or flicker over long rollouts
- **x-prediction is a denoising target:** the network only needs to know what the clean frame should look like — errors are 'pulled back' toward plausible images every step
- **Combined with noised context ($\tau_{\text{ctx}} \approx 0.1$):** the model is trained and run on slightly corrupted context, so it never sees a distribution it was not trained on
- **Evidence:** full architecture with v-space prediction/loss reaches FVD 124; with x-prediction and x-space loss, 57 (Table 2). Long 384-frame generations were used for FVD precisely to expose compounding error

Generation quality for sampling steps

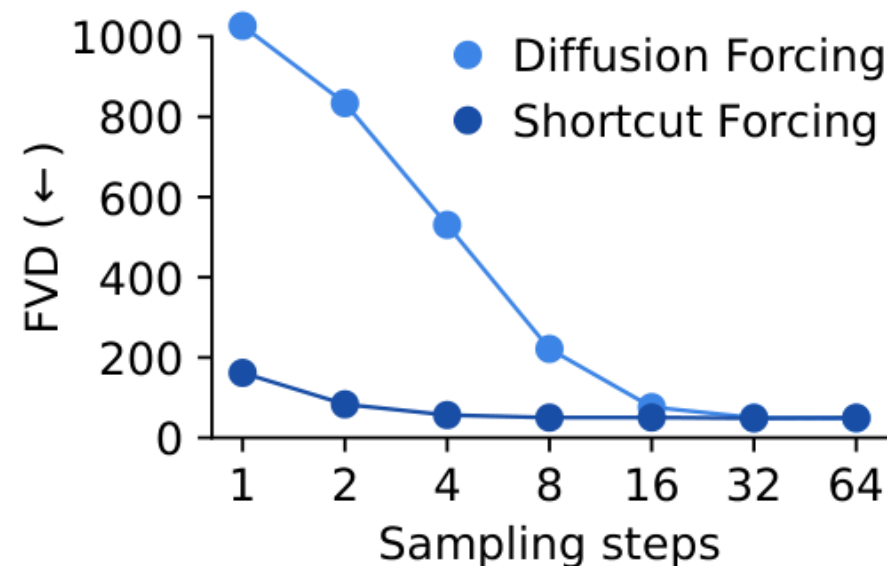
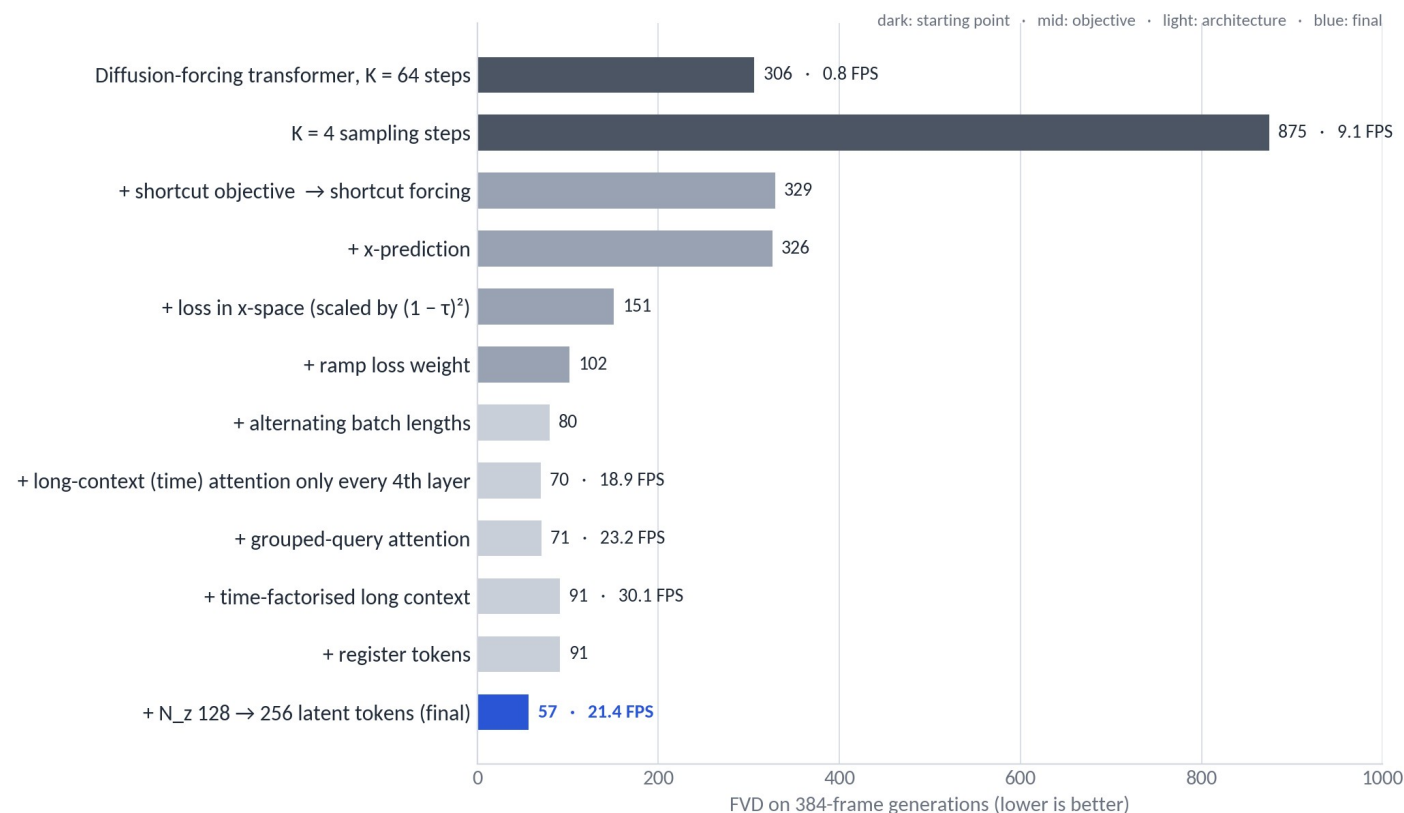


Figure 8: FVD vs sampling steps. Shortcut forcing at 4 steps ≈ 60 ; diffusion forcing needs 16–32 steps to match.

Architecture choices that buy speed without losing quality

Table 2: each row adds one change to the previous row



- **Alternating batch lengths** (64 / 256 frames): FVD 102 → 80 and a training step from 9.8 s → 1.5 s
- **Time attention only every 4th layer**: 18.9 FPS at FVD 70
- **Grouped-query attention**: 23.2 FPS, FVD unchanged
- **Time-factorised long context** trades a little quality (91) for 30 FPS; **registers** stabilise attention
- **More latent tokens** (128 → 256) brings FVD to 57 at 21.4 FPS — the final configuration
- Runs were 48 h each; FVD computed on 1,024 generations of 384 frames

From world model to agent: agent tokens & heads

Phase 2 — policy and reward live inside the same transformer, but cannot disturb the simulation

- **Agent tokens:** extra tokens per frame that carry a task embedding (one-hot over the 20 training tasks) and read out policy and reward
- **One-way attention:** agent tokens attend to actions, latents and past frames — but *nothing attends back to them*, so the world model never conditions on the policy's intentions or reward estimates
- **Policy head — behavioural cloning with multi-token prediction:** predict the next $L = 8$ actions; the extra horizon regularises the representation and encodes intent
- **Reward head:** predicts task rewards derived from game events (item obtained); a task-conditioned *scalar* reward was chosen over a reward vector so that open-vocabulary tasks remain possible
- **Value head:** added in Phase 3, trained by TD learning on imagined rollouts
- **Result already at this stage:** 'WM + BC' beats BC on every hard milestone (iron pickaxe 16.9 % vs 0.6 %) — the video-prediction representation is a strong prior for control

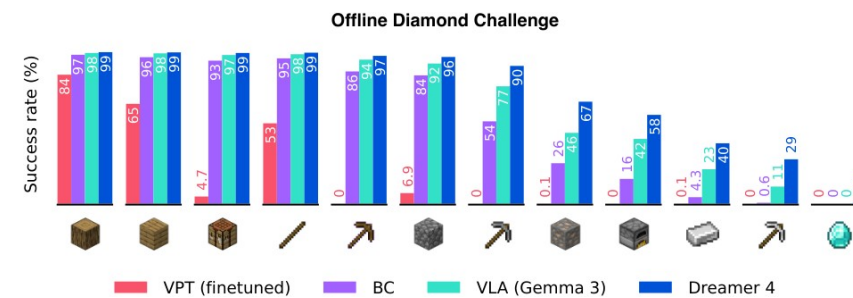


Figure 3: success rates of agents trained on the same offline data (BC, VLA, WM+BC, Dreamer 4).

WHY NOT A SEPARATE POLICY NETWORK?

Sharing the backbone gives the policy the world model's representation for free, and keeps everything one model. The one-way attention mask is what makes this safe.

Imagination training with PMPO

Phase 3 — RL on imagined rollouts, with the transformer frozen

- **Rollouts:** start from real context frames, let the policy act, and let the world model generate what happens next; rewards come from the learned reward head; $\gamma = 0.997$
- **Frozen transformer:** only policy and value heads are trained → the simulator cannot drift toward the policy
- **PMPO (Preference-style MPO):** instead of scaling the policy gradient by the advantage, use only its *sign* — positive-advantage actions are pushed up, negative ones down, with balanced weight $\alpha = 0.5$ on each set
- **Reverse-KL to the frozen BC policy** with $\beta = 0.3$ keeps the policy on the data manifold — analogous to KL-regularised RLHF; also the main guard against exploiting world-model gaps
- **Why not Dreamer 3's recipe?** Return normalisation + entropy bonus were replaced by the sign-based update: robust to reward scale and to the extremely sparse late-game rewards
- **Effect:** iron pickaxe 16.9 % → 29.0 %, iron ingot 27.8 % → 39.5 %, diamond 0 → 0.7 %; agents also become faster (Table 8)



Imagined 'mine stone' rollout with reward and value traces (project page).

FAILURE MODE THE KL TERM ADDRESSES

Hafner's example: the policy discovers it can 'craft' a pickaxe from invalid materials because the model renders it anyway. The KL to the BC prior limits such exploitation.

PART 3

Experiments

Real-time play-testing, the offline diamond challenge, ablations, label efficiency, robotics

03

Is the world model accurate? Let humans play inside it

Table 1: real-time interaction on one H100; 16 tasks attempted by human testers from the same starting frame

Model	Params	Resolution	Context	FPS	Tasks solved
MineWorld	1.2 B	384×224	0.8 s	2	not evaluable (needs actions in advance)
Lucid-v1	1.1 B	640×360	1.0 s	44	0 / 16
Oasis (small)	500 M	640×360	1.6 s	20	0 / 16
Oasis (large)	—	360×360	1.6 s	~5	5 / 16
Dreamer 4	2 B	640×360	9.6 s	21	14 / 16

- **The 16 tasks:** eat apple · place 3 torches · chop tree · dig 3×3 pit · craft wooden pickaxe · use furnace · place & open workbench · kill zombies · mine diamonds · complete window · place & ride boat · enter portal · place door · bed & sleep · plant reed & pour water · turn 360° & enter house
- **The two failures** were memory-related: the world changes when you look away for longer than the 9.6 s context, and inventory contents become unclear over time
- Genie 3 is not compared: it has no fine-grained mouse/keyboard control



Figure 5: same task, same start frame — Dreamer 4 (top) builds the wall; Lucid-v1 and Oasis do not.

WHY THIS EVALUATION

FVD and PSNR measure how video *looks*; only interaction tests whether the model got the *mechanics* right. The paper complements it with FVD on 384-frame rollouts, but no long-horizon *quantitative* interaction metric exists yet — a fair critique.

What 'accurate object interactions' looks like

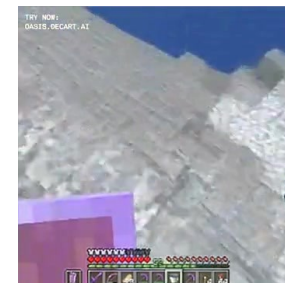
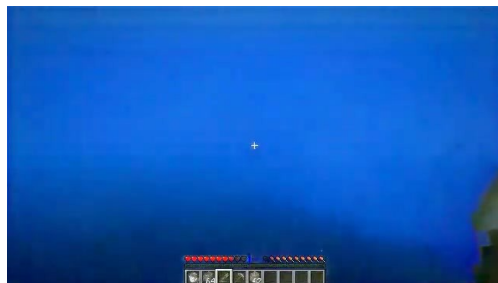
Frames from the project page's side-by-side human sessions (same start frame, same human inputs)

Dreamer 4

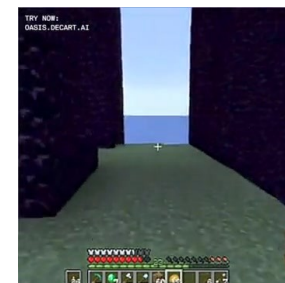
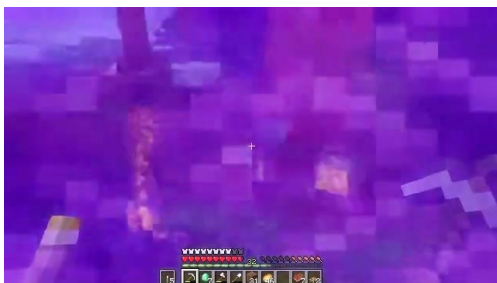
Lucid-v1

Oasis

Place and ride
boat



Enter portal



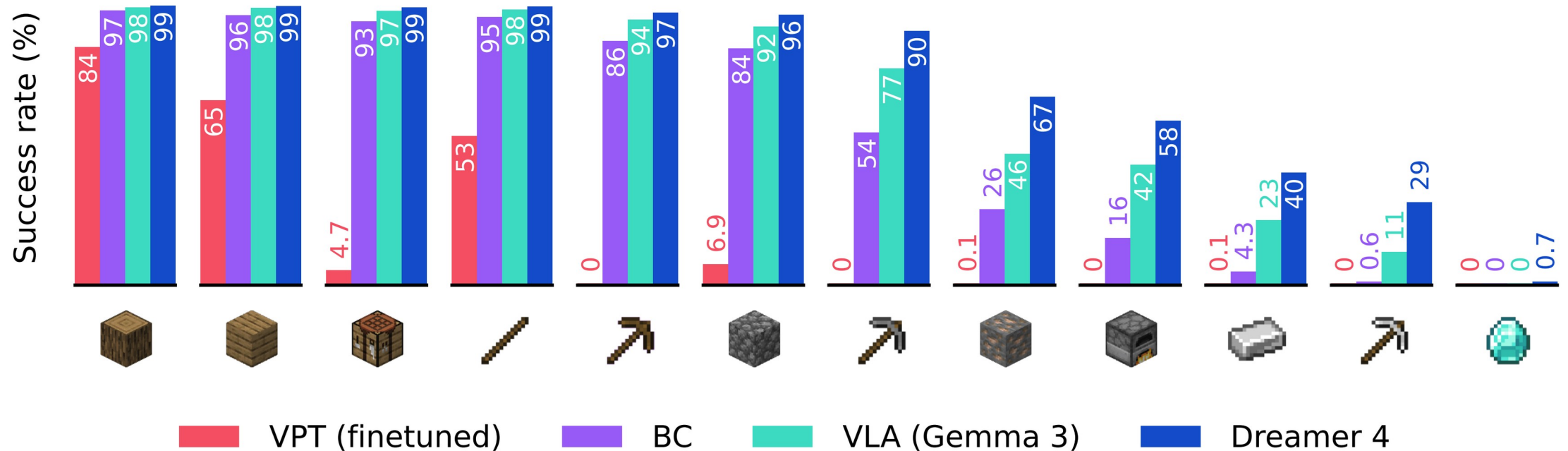
Craft wooden
pickaxe



Single frames sampled from the project-page clips (danijar.com/asset/dreamer4/human/...). Dreamer 4 renders the boat ride, the Nether-portal swirl and a working crafting-table menu; the baselines lose the scene or the menu.

Offline diamond challenge: results

Success rate per milestone; 1,000 episodes × 60 min; same 2,541 h of data for all methods (Table 7 / Fig. 3)

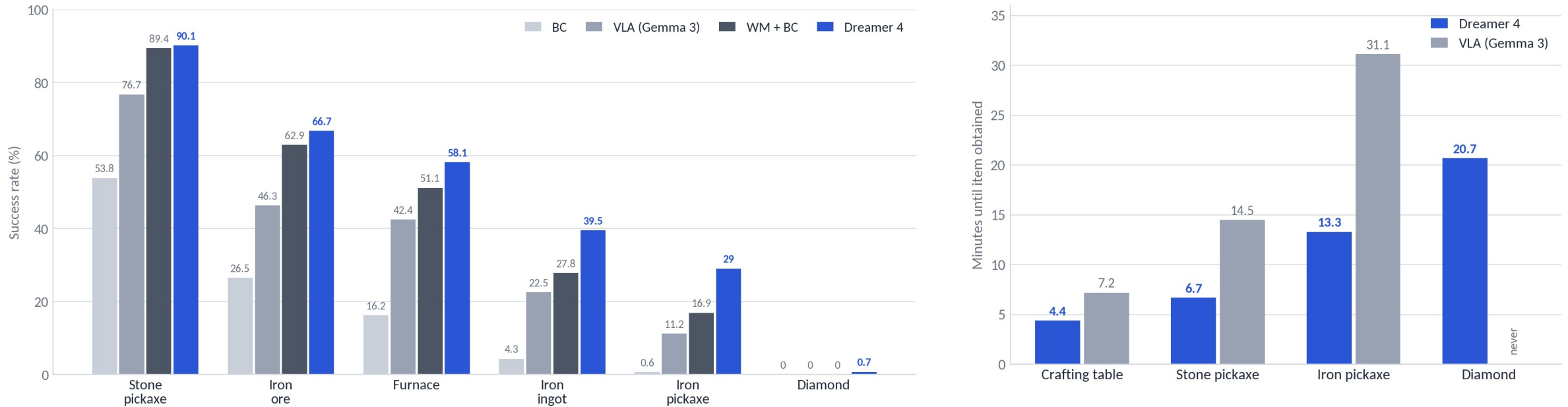


Official figure from danijar.com/project/dreamer4 (benchmark.png).

- **Dreamer 4:** stone pickaxe 90.1 % · iron ore 66.7 % · furnace 58.1 % · iron ingot 39.5 % · iron pickaxe 29.0 % · **diamond 0.7 %** (≈ 7 of 1,000 episodes)
- **VLA on Gemma 3 (same data):** iron pickaxe 11.2 %, no diamonds · **BC:** 0.6 % · **VPT fine-tuned** (pretrained on 270 K h of web video): only reliably reaches sticks
- Gap widens along the tech tree: imitation is enough for early items, imagination RL matters where data is thin and horizons are long

Where do the gains come from? Agent ablations and speed

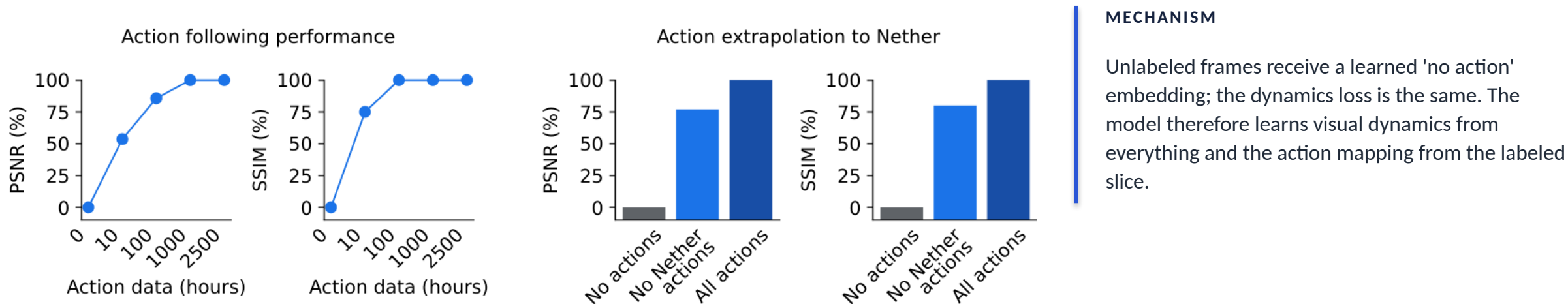
Figure 4 / Table 8: representation (WM+BC vs BC) and imagination RL (Dreamer 4 vs WM+BC) both contribute



- **Representation:** BC on the world model's features (WM+BC) vs plain BC — iron pickaxe 16.9 % vs 0.6 %; authors: 'video prediction implicitly learns an understanding of the world that is useful for decision making'
- **Imagination RL:** adds +12 pts on iron pickaxe and +12 pts on iron ingot over WM+BC, and unlocks diamonds
- **Speed:** Dreamer 4 is fastest at every milestone — iron pickaxe in 13.3 min vs 31.1 min for the VLA; diamonds in 20.7 min on average when achieved (human average $\approx$ 20 min)
- **Task input matters:** BC without the task embedding collapses to 8.8 % stone pickaxe (Table 7)

Unlabeled video: how many action labels are needed?

Figure 7 — action conditioning from a small labeled fraction, and transfer to unseen visual domains



- **Setup:** train on all 2,541 h of video, but keep actions for only a subset; measure PSNR/SSIM of action-conditioned predictions, normalised between 'no actions' (0 %) and 'all actions' (100 %)
- **10 h of labels** → 53 % PSNR / 75 % SSIM; **100 h** → 85 % / 100 %; **1,000 h** → 100 % — action understanding saturates with roughly 4 % of the data labeled
- **Domain transfer:** with actions only from the Overworld, predictions in the Nether/End (seen unlabeled) recover 76 % / 80 % — the model separates 'what actions do' from 'what the world looks like'
- **Why it matters:** the road to internet-scale video pretraining, where labels are scarce (VPT needed an inverse-dynamics model for this)

Inside the imagination: decoded training rollouts

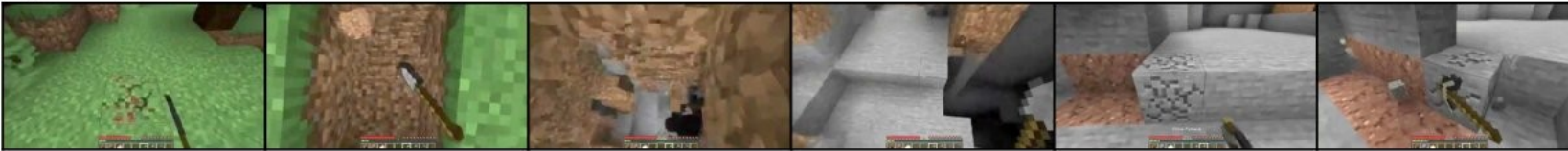
Figure 1 — imagined sequences for four agent tasks, decoded only for visualisation

Imagination Training Inside the World Model

Agent task: Gather wood



Agent task: Mine cobblestone



Agent task: Craft stone pickaxe



Agent task: Mine diamonds



Time →

- The world model simulates chopping, crafting menus, mining, cave lighting and diamond ore — all generated, not recorded
- RL never sees pixels: it operates on latents; decoding is for us
- Each rollout starts from real context frames and follows the *current* policy → training data stays on-policy without touching the game
- Stochastic sampling gives diverse futures — useful for exploring rare outcomes such as finding ore

Beyond Minecraft: robotics & egocentric video (qualitative)

Figure 6 and Figure 9 — same recipe, different data; no quantitative benchmark yet



Figure 6: SOAR robot-arm data — counterfactual actions (pick objects, flip bowl, press ball, move towel, throw bowl) rendered by the world model.



Project page: human counterfactual interaction with the robot world model.

- **SOAR:** 180 h of teleoperation + online-RL data, 256×256 at 5 FPS, 7-D relative end-effector actions; $N_z = 512$, context 96 frames
- **Epic Kitchens 100:** 100 h of egocentric kitchen video, no actions — generations from held-out context (Fig. 9) show the recipe works on real, unstructured video
- **Caveat:** no policy was trained and no metrics are reported for these domains — they demonstrate generality of the world model, not of the agent

Dreamer 3 vs Dreamer 4

Appendix F: what changed, and what the higher-fidelity model costs

	Dreamer 3 (Nature 2025)	Dreamer 4
World model	RSSM (recurrent + variational latent)	Block-causal diffusion/flow transformer, shortcut forcing
Minecraft input	64×64 pixels + inventory state; abstract crafting actions	360×640 pixels only; raw mouse & keyboard
Data	1.4 K h of <i>online</i> interaction, no human data	2.5 K h of <i>offline</i> human data, no interaction
RL	Return normalisation + entropy bonus	PMPO (advantage sign) + KL to BC prior
Speed	~1,000× faster than a diffusion transformer	~30× faster than diffusion forcing; ~40× slower than the RSSM

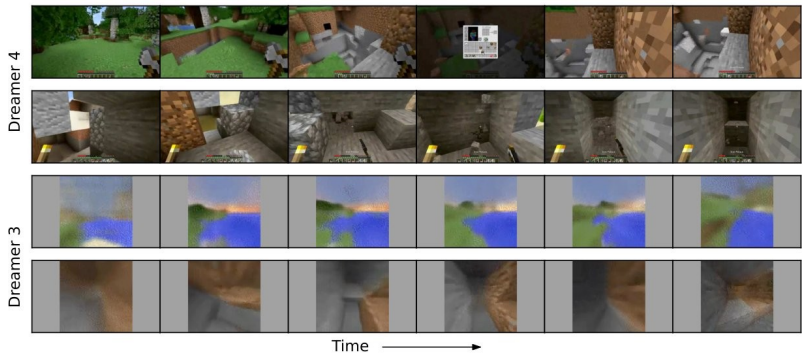


Figure 11: multi-step generations, Dreamer 4 (top) vs Dreamer 3 (bottom).



Generation speed on a log scale (paper, TalkRL): ~30× faster than diffusion forcing, ~40× slower than the RSSM.

Data, compute and scale

Appendix A and Table 3

Dataset	Size	Resolution / FPS	Actions	Dynamics config
Minecraft (VPT contractor 6–10)	2,541 h	360×640 → 384×640 padded, 20 FPS	23 keys + 121-way mouse; item events → rewards	N_z 256 · C 192 · T 64/256
Overworld vs Nether/End split	subset	same	labels only for Overworld	action-generalisation study
SOAR robotics	180 h	256×256, 5 FPS	7-D relative end-effector	N_z 512 · C 96 · T 32/128
Epic Kitchens 100	100 h	256×256, 10 FPS	none	same as SOAR

Agent (Table 3)	Human data	Web video	Online interaction
VPT (RL fine-tuned)	2.5 K h	270 K h	194 K h
VPT (BC)	2.5 K h	270 K h	—
Dreamer 3	—	—	1.4 K h
Dreamer 4	2.5 K h	—	—

- **Compute:** 256–1,024 TPU v5p chips per run (exact chip-hours not disclosed)
- **Ablations:** 48 h runs per configuration
- **Model:** ≈ 2 B parameters total; inference 21 FPS on one H100
- **Not released:** code, weights or data recipe (Hafner: 'probably not') — reproductions exist (Open Dreamer, Dreamer-MC, Hansen's PyTorch port)

PART 4

Discussion

Limitations, critical reading, future work, takeaways

04

Limitations the paper itself states

From the discussion section and the authors' talks

- 01 **Diamonds are rare:** 0.7 % success; the authors call the pipeline a 'reliable and performant starting point', not a solved task
- 02 **Memory:** 9.6 s of context; walk into a house for 30 s and the outside is regenerated — long-term memory is the number-one item on the roadmap
- 03 **Inventory drift:** small UI elements and item counts blur over long horizons; small objects and long-range semantic correlations are what video models learn last
- 04 **Frozen, exploitable simulator:** no epistemic-uncertainty estimate; the policy can exploit model errors (invalid crafting) — currently mitigated only by the KL constraint
- 05 **Two-stage training:** tokenizer is trained separately and frozen; the authors expect end-to-end training eventually
- 06 **Single domain for the agent:** RL results exist only for Minecraft; robotics and kitchen video are qualitative world-model demos
- 07 **Compute:** hundreds of TPU v5p chips per run; not a recipe individuals can reproduce at full scale

A critical reading: established vs. open

What a reviewer would push on (and what independent groups later found)

Well supported

- Shortcut forcing + x-prediction gives a large speed/quality win — a clean 12-step ablation cascade, and reproduced by Open Dreamer at 1.6 B scale (JAX, 2026).
- The world model handles object interactions far better than Oasis / Lucid-v1 — consistent human play-tests plus long-rollout FVD.
- World-model features are a strong prior for imitation (WM+BC $\gg$ BC on the same data).
- Action conditioning from ~4 % labeled data works and transfers across visual domains.

Open or thinly supported

- Diamond result: 0.7 % without confidence intervals or per-seed variance; success criterion is a game event, protocol details are in the appendix only.
- No direct quantitative long-horizon accuracy metric — interaction accuracy is human-judged on 16 tasks (Pith/EmergentMind critique).
- RL gains over WM+BC are modest on most items; small-scale reproductions report high seed variance in the RL phase and reward-hacking of the frozen reward head.
- No peer review as of Sept 2026 (arXiv v1 only); no code or weights; nobody outside DeepMind has reproduced offline diamonds.
- Generality of the *agent* beyond Minecraft is untested.

Where the authors want to take it

Future work listed in the paper and expanded in the TalkRL interview (Nov 2025)

01

Internet-video pretraining

Learn dynamics from unlabeled web video; label only a small slice with actions (Fig. 7 is the proof of concept).

02

Long-term memory

Beyond 9.6 s: retrieval or compressed memory so the world stays consistent when you look away.

03

Language conditioning

Replace one-hot task IDs with text embeddings; scalar task-conditioned reward already anticipates open-vocabulary tasks.

04

A little online data

Small rounds of *corrective* interaction to fix the errors the policy exploits — then the KL constraint can be relaxed.

05

Automatic goal discovery

Empowerment / exploration objectives (APD, Plan2Explore, Director) so agents propose their own tasks inside the model.

06

End-to-end training

Fold the tokenizer into the dynamics model; scale parameters and context together.

Key takeaways

01

Imagination training works offline, from pixels

Dreamer 4 reaches diamonds in Minecraft with zero environment interaction and about 100× less data than VPT.

04

Video pretraining is a strong prior for control

Most of the gain over BC comes from the world-model representation; RL adds the rest on the hardest, longest-horizon items.

02

The enabling contribution is speed at quality

Shortcut forcing (4 sampling steps), x-prediction with an x-space loss, and time attention only in every 4th layer: 21 FPS on one GPU with a 9.6 s context.

05

Action labels are cheap

About 100 h of labeled actions on top of thousands of unlabeled hours suffice, and action semantics transfer to unseen visual domains.

03

Interaction accuracy, not visual polish, is the bar

Validated by humans playing inside the model: 14 of 16 tasks, against 5 of 16 for the best prior Minecraft world model.

06

Open problems remain

Short memory, exploitable frozen simulators, thin statistics on the headline result, and no official code or weights.

Questions for discussion

- 01 How much of the diamond result is the world model versus PMPO? Would a stronger offline-RL baseline on world-model features (no imagination) close the gap?
- 02 Is human play-testing on 16 tasks a sufficient measure of 'accurate simulation'? What automated long-horizon metric would you propose?
- 03 The transformer is frozen during RL and the reward head is learned: where would you expect exploitation to appear first, and how would you detect it offline?
- 04 9.6 s of context: which memory mechanism (retrieval, state-space layers, compressed summaries) fits a block-causal diffusion transformer best?
- 05 The recipe needs ~100 h of action labels per domain. What does the path to internet-scale video look like — inverse dynamics models, or Dreamer 4's 'no-action' embedding at scale?
- 06 What would convince you that this transfers to real robots — and what evaluation would you run first?

References & resources

- **Paper:** Hafner*, Yan*, Lillicrap. Training Agents Inside of Scalable World Models. arXiv:2509.24527 (29 Sep 2025). <https://arxiv.org/abs/2509.24527>
- **Project page & videos:** <https://danijar.com/project/dreamer4/> — overview video (youtube.com/watch?v=oDlBtTcX0g0), four uncut 60-min 'Diamond Challenge' evaluation episodes, 51 human-interaction clips
- **Authors in conversation:** TalkRL E73 'Danijar Hafner on Dreamer v4' (10 Nov 2025), transcript available; Hack Club AMA (Dec 2025)
- **Ingredients:** Chen et al., Diffusion Forcing (2407.01392) · Frans, Hafner, Levine, Abbeel, Shortcut Models (2410.12557) · Chen et al., MAETok (2502.03444) · Baker et al., VPT (2206.11795) · Hafner et al., DreamerV3 (2301.04104)
- **Independent reproductions / follow-ups:** next-state/open-dreamer (JAX, 1.6 B Minecraft world model, Jul 2026) · lamCreateAI/Dreamerv4-MC (weights on HF) · nicklashansen/dreamer4 & MMBench2 (Hansen & Wang, 2606.27326) · lucidrains/dreamer4 (PyTorch)
- **Everything above, archived:** <https://github.com/HarmannBitte/Dreamer-4> — PDFs, repo snapshots, articles, transcripts, the official clips used in this deck, and a MEDIA_INDEX.md

Appendix A — Configuration & hyper-parameters

Component	Setting
Tokenizer	Masked autoencoder; 16×16 patches; patch dropout $\sim U(0, 0.9)$; loss MSE + 0.2·LPIPS; bottleneck 512×16 with tanh $\rightarrow N_z = 256 \text{ tokens} \times 32 \text{ dims}$; causal in time; $\approx 400 \text{ M params}$; frozen after pretraining
Dynamics model	$\approx 1.6 \text{ B params}$; pre-RMSNorm, RoPE, SwiGLU, QK-norm, logit soft-capping, register tokens; space attention in 3/4 layers, causal time attention every 4th layer; GQA; context C = 192 frames (9.6 s); alternating batch lengths T1/T2 = 64/256
Shortcut forcing	per-frame signal level τ , step size d; x-prediction; bootstrap target formed in v-space, loss scaled to x-space by $(1-\tau)^2$; ramp weight $w(\tau) = 0.9\tau + 0.1$; K = 4 sampling steps; context noise $\tau_{\text{ctx}} \approx 0.1$
Actions	23 binary keys + 121-way mouse (μ -law, 11×11 foveated bins, VPT convention); learned 'no action' embedding for unlabeled video
Agent (phase 2)	agent tokens with one-hot task (20 tasks), one-way attention; BC with multi-token prediction L = 8; task-conditioned scalar reward head; data mix 50 % uniform / 50 % task-relevant
Imagination RL (phase 3)	transformer frozen; policy + TD value heads; PMPO with advantage sign, $\alpha = 0.5$, reverse-KL $\beta = 0.3$ to frozen BC policy; $\gamma = 0.997$
Evaluation	1,000 episodes $\times$ 60 min, empty inventory, random worlds, fixed prompt sequence; human interaction: 16 tasks on one H100 at 21 FPS
Compute / data	256–1,024 TPU v5p; 2,541 h VPT contractor video (subsets 6–10), 90/10 split by 5-min chunk; ablations: 48 h runs, FVD on 1,024 $\times$ 384-frame generations

Appendix B — Full design cascade (Table 2)

Change (cumulative)	FPS	FVD ↓
Diffusion-forcing transformer, K = 64	0.8	306
K = 4 sampling steps	9.1	875
+ shortcut objective (→ shortcut forcing)		329
+ x-prediction		326
+ x-space loss		151
+ ramp weight		102
+ alternating batch lengths (step 9.8 s → 1.5 s)		80
+ long context only every 4th layer	18.9	70
+ grouped-query attention	23.2	71
+ time-factorised long context	30.1	91
+ register tokens		91
N_z 128 → 256 (final)	21.4	57
(reference) full architecture, v-space prediction & loss		124

- 48-hour training runs per row
- FVD on 1,024 generations of 384 frames (long rollouts to expose compounding error)
- FPS measured for generation on one H100 where reported
- Fig. 8 adds the steps sweep: shortcut forcing FVD $\approx$ 160/85/60/50 at 1/2/4/8 steps vs $\approx$ 1,020/830/530/220 for diffusion forcing

Appendix C — Full success-rate table (Table 7, %)

Milestone	VPT (ft)	BC (no task)	BC	VLA (Gemma 3)	WM + BC	Dreamer 4
Log	84.3	71.4	97.3	98.5	99.6	99.1
Planks	65.3	68.6	95.7	98.3	99.6	98.9
Crafting table	4.7	63.8	93.5	97.2	99.1	98.5
Stick	52.6	62.4	95.0	97.7	98.9	98.7
Wooden pickaxe	0.0	33.8	86.5	94.1	97.3	96.6
Cobblestone	6.9	32.0	83.9	91.6	97.2	95.9
Stone pickaxe	0.0	8.8	53.8	76.7	89.4	90.1
Iron ore	0.1	3.6	26.5	46.3	62.9	66.7
Furnace	0.0	4.0	16.2	42.4	51.1	58.1
Iron ingot	0.1	0.2	4.3	22.5	27.8	39.5
Iron pickaxe	0.0	0.0	0.6	11.2	16.9	29.0
Diamond	0.0	0.0	0.0	0.0	0.0	0.7

1,000 evaluation episodes of 60 minutes each; all agents trained on the same 2,541 h of contractor data (VPT additionally pretrained on 270 K h of web video).

Appendix D — Glossary

Term	Meaning in this paper
World model	Learned model that predicts future observations (here: latent frames) given past observations and actions
Imagination training	Training the policy on rollouts generated by the world model instead of the real environment
Tokenizer / latents	Encoder–decoder mapping each frame to 256 continuous tokens the dynamics model operates on
Block-causal attention	Full attention within a frame, causal across frames — enables frame-by-frame online generation
Diffusion forcing	Diffusion training with an independent noise level per frame in a sequence
Shortcut model	Diffusion/flow model conditioned on step size, trained to be self-consistent across step sizes → few-step sampling
x- vs v-prediction	Network predicts the clean latent (x) vs the velocity $x - \epsilon(v)$; Dreamer 4 uses x-prediction
FVD / PSNR / SSIM	Fréchet Video Distance (distribution-level video quality, lower is better) / per-frame reconstruction fidelity metrics
GQA, RoPE, registers	Grouped-query attention (fewer KV heads), rotary position embeddings, extra learnable tokens that absorb attention mass
BC, VLA, PMPO	Behavioural cloning; vision-language-action model (here built on Gemma 3); the sign-based, KL-regularised policy-gradient update used for RL